

RONB EVENTS

Corporate Championship

Complete Competition-System Implementation Plan

System objective

Add a tournament engine to ronbevents.com without replacing the existing registration, payment, team, player, authentication, or email systems.

Prepared: 2 October 2026

1. Executive Summary

The recommended architecture keeps the current RONB Events registration system as the source of participating companies and adds a separate competition layer for groups, fixtures, results, standings, qualification, and knockout progression.

The core design principle is simple: registration determines who is participating; competition determines what happens to them after they are confirmed.

Event -> Sports -> Orders -> Order Items -> Confirmed Registration -> Competition Entry -> Groups -> Fixtures -> Matches -> Results -> Standings -> Knockout -> Champion

Sport	Companies	Groups	Per Group	Group Matches	Qualifiers/Group
Futsal	32	8	4	48	2
Cricksal	20	4	5	40	2
Basketball	16	4	4	24	2
TOTAL	68	16	-	112	-

Assuming the top two companies from every group qualify, the knockout stage becomes 16-team for futsal and 8-team for both cricksal and basketball. With no third-place matches, that produces 29 knockout matches and 141 matches overall. With three third-place matches, the overall total would be 144.

2. Existing System and Integration Boundary

The current schema already stores company/team name and logo on each sport registration in order_items, while team_players are linked to the order_item. That means the competition engine does not need to rebuild registration or player storage.

orders -> order_items -> team_players

confirmed order_item -> competition_team -> competition data

Terminology

Public UX should generally say Company, Company Logo, Company Entry, Players, Match, Group, Standing, and Champion. Internally, competition_teams is still a sensible technical name for the participant entity.

Do not create a global companies table for the first version unless RONB needs cross-sport company profiles or historical company statistics. The tournament mechanics work without one.

3. Target Architecture



```

+-- GROUP
+-- FIXTURES
+-- MATCHES
+-- RESULTS
+-- STANDINGS
+-- QUALIFICATION
+-- KNOCKOUT
+-- CHAMPION

```

- Keep registration, payment verification, receipts, and existing user/admin authentication unchanged.
- Treat confirmed order_items as the source of eligible company entries.
- Freeze company name/logo in the competition entry as a tournament snapshot so later registration edits cannot rewrite history.
- Make the backend authoritative for scores, winners, points, standings, qualification, and bracket progression.
- Keep sport rules configurable rather than scattering sport-specific if/else logic across route handlers.

4. Database Design

4.1 competitions

```

id UUID
event_id UUID NOT NULL
name TEXT NOT NULL
slug TEXT NOT NULL UNIQUE
status TEXT NOT NULL
start_date TIMESTAMPTZ
end_date TIMESTAMPTZ
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL

```

Recommended status values: draft, setup, groups_generated, fixtures_generated, live, group_stage_complete, knockout, completed, cancelled.

4.2 competition_sports

```

id UUID
competition_id UUID NOT NULL
sport_id UUID NOT NULL
format TEXT NOT NULL
group_count INT NOT NULL
teams_per_group INT NOT NULL
qualifiers_per_group INT NOT NULL
win_points NUMERIC NOT NULL
draw_points NUMERIC NOT NULL
loss_points NUMERIC NOT NULL
allows_group_draw BOOLEAN NOT NULL
group_tie_resolution TEXT NOT NULL
knockout_tie_resolution TEXT NOT NULL
rules JSONB NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL

```

4.3 competition_groups

id UUID
 competition_sport_id UUID NOT NULL
 name TEXT NOT NULL
 position INT NOT NULL
 created_at TIMESTAMPTZ NOT NULL
 UNIQUE(competition_sport_id, name)

4.4 competition_teams

id UUID
 competition_sport_id UUID NOT NULL
 order_item_id UUID NOT NULL
 group_id UUID
 team_name_snapshot TEXT NOT NULL
 logo_url_snapshot TEXT
 seed INT
 group_position INT
 status TEXT NOT NULL
 created_at TIMESTAMPTZ NOT NULL
 updated_at TIMESTAMPTZ NOT NULL
 UNIQUE(competition_sport_id, order_item_id)

Recommended status values: registered, active, eliminated, qualified, withdrawn, disqualified, champion.

4.5 matches

id UUID
 competition_sport_id UUID NOT NULL
 stage TEXT NOT NULL
 group_id UUID
 home_team_id UUID
 away_team_id UUID
 scheduled_at TIMESTAMPTZ
 venue TEXT
 court TEXT
 status TEXT NOT NULL
 home_regulation_score INT
 away_regulation_score INT
 home_final_score INT
 away_final_score INT
 winner_team_id UUID
 tie_resolution TEXT
 round_number INT
 bracket_position INT
 source_home_match_id UUID
 source_away_match_id UUID
 fixture_key TEXT
 notes TEXT
 created_at TIMESTAMPTZ NOT NULL
 updated_at TIMESTAMPTZ NOT NULL
 completed_at TIMESTAMPTZ

Use a distinct fixture_key and a uniqueness constraint so fixture generation is idempotent and cannot create duplicate pairings.

4.6 match_periods

```
id UUID
match_id UUID NOT NULL
period_type TEXT NOT NULL
period_number INT
home_score INT NOT NULL
away_score INT NOT NULL
metadata JSONB
created_at TIMESTAMPTZ NOT NULL
```

period_type can include regulation, overtime, super_over, and penalty.

4.7 competition_qualifiers

```
id UUID
competition_sport_id UUID NOT NULL
stage TEXT NOT NULL
team_id UUID NOT NULL
group_id UUID
group_position INT
seed INT
qualification_reason TEXT
created_at TIMESTAMPTZ NOT NULL
```

4.8 match_result_versions

```
id UUID
match_id UUID NOT NULL
old_data JSONB NOT NULL
new_data JSONB NOT NULL
changed_by UUID NOT NULL
reason TEXT NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

This preserves corrections without deleting previous results. The existing audit_logs table should also receive an audit entry for every correction.

5. Sport Rules Configuration

5.1 Futsal

```
{
  "group_count": 8,
  "teams_per_group": 4,
  "qualifiers_per_group": 2,
  "points": { "win": 3, "draw": 1, "loss": 0 },
  "allows_group_draw": true,
  "group_tie_resolution": "draw",
  "knockout_tie_resolution": "penalties"
}
```

A group match such as 2-2 awards 1 point to each company. A knockout match cannot finish as an unresolved draw; the configured penalty procedure determines the winner.

5.2 Cricksal

```
{
  "group_count": 4,
  "teams_per_group": 5,
  "qualifiers_per_group": 2,
  "points": { "win": 2, "loss": 0 },
  "allows_group_draw": false,
  "group_tie_resolution": "super_over",
  "knockout_tie_resolution": "super_over"
}
```

If the regulation score is tied, the backend requires a recorded Super Over result before the match can become completed.

If official standings use Net Run Rate, the database must store enough innings detail to calculate NRR correctly. Do not calculate NRR from incomplete score data.

5.3 Basketball

```
{
  "group_count": 4,
  "teams_per_group": 4,
  "qualifiers_per_group": 2,
  "points": { "win": 2, "loss": 0 },
  "allows_group_draw": false,
  "group_tie_resolution": "overtime",
  "knockout_tie_resolution": "overtime"
}
```

A regulation tie such as 72-72 requires overtime. The final score becomes the official match score and the winner becomes the knockout/group result.

6. Standings and Tie-Breaking Engine

Do not initially store standings as a manually editable table. Calculate them from completed group matches. A standings cache can be added later if traffic requires it.

```
calculateStandings(competitionSportId, groupId)
-> load teams
-> load completed group matches
-> aggregate statistics
-> apply points
-> apply sport-specific statistics
-> apply configured tie-break rules
-> assign position
-> return ordered standings
```

The actual tie-break order must be confirmed with RONB before development is finalized. The engine should support an ordered configuration such as points -> goal difference -> goals for -> head-to-head -> fair play -> draw lots, or sport-specific alternatives such as points -> net run rate -> head-to-head -> draw lots.

7. Group Draw and Fixture Generation

7.1 Group draw

1. Import only confirmed registrations.
2. Validate the team count against the configured group format.
3. Generate a reproducible draw using a server-side seed.
4. Allow admins to review the generated groups.
5. Allow manual movement before fixtures exist.
6. Confirm the group draw and lock group assignments.

Save the draw seed and record the draw action in audit_logs so the assignment can be reproduced and reviewed later.

7.2 Round-robin fixture generation

For each group:

for every unordered pair of teams:
create one fixture

4 teams -> 6 fixtures

5 teams -> 10 fixtures

Sport	Groups	Fixtures/Group	Total
Futsal	8	6	48
Cricksal	4	10	40
Basketball	4	6	24

Fixture generation must guarantee no self-play, no duplicate pair, every pair exactly once, correct group membership, and exact expected fixture count. Use a deterministic fixture_key plus a unique constraint.

8. Match Lifecycle and Result Processing

scheduled -> live -> completed

|

+> postponed

+> cancelled

+> abandoned

8.1 Result submission

7. Lock the match row with SELECT ... FOR UPDATE inside a database transaction.
8. Validate the match status and participating company IDs.
9. Validate score ranges and sport-specific rules.
10. Record regulation and resolution periods where applicable.
11. Determine winner/draw and final score on the server.
12. Update the match and completion timestamp.
13. Write an audit log and, where relevant, result-version history.
14. Advance a knockout slot in the same transaction when the winner is known.

Frontend rule

The browser sends raw score data. It never sends authoritative points, standings, winner IDs, qualifier IDs, or bracket winners.

8.2 Futsal example

Group: 2-2 -> draw = true -> 1 point each

Knockout: 2-2 -> require penalty result -> winner_team_id = penalty winner

8.3 Cricksal example

Regulation: 86-86 -> not completed

Super Over: 12-9 -> winner = 12-run side -> completed

8.4 Basketball example

Regulation: 72-72 -> require overtime

Final: 81-78 -> winner = 81-point side -> completed

9. Corrections, Concurrency, and Auditability

Result correction is a first-class feature. Never delete a completed result just to replace it.

```
original result
-> corrected result
-> latest official result
```

Require a reason for completed-result corrections. Use a super_admin permission for changes that may alter qualification or an existing knockout bracket.

9.1 Concurrency

Two administrators may open the same match simultaneously. Protect result writes with row locking and optimistic version checks. A stale browser submission should receive a 409 Conflict response and refresh rather than silently overwriting a newer result.

```
{ "version": 12, "homeScore": 2, "awayScore": 0 }
```

9.2 Audit actions

- competition.created / updated
- sport configuration updated
- groups.generated / updated / confirmed
- fixtures.generated
- match.started / completed / postponed / cancelled
- match.result.corrected
- knockout.generated
- competition.stage_changed

10. Qualification and Knockout Engine

10.1 Qualification

15. Verify every required group fixture has a terminal result.

16. Calculate standings.

17. Apply all configured tie-breakers.
18. Select the configured number of qualifiers per group.
19. Create competition_qualifiers records with source group and position.
20. Assign knockout seeds.

10.2 Knockout sizes

Sport	Qualified	First KO Round	QF	SF	Final
Futsal	16	R16: 8	4	2	1
Cricksal	8	QF: 4	-	2	1
Basketball	8	QF: 4	-	2	1

10.3 Bracket sources

Knockout slots should be sourced from previous match IDs, not manually assigned after every result. When a source match completes, its winner automatically populates the appropriate next-round slot.

QF1 winner -> SF1 home
 QF2 winner -> SF1 away
 SF1 winner -> FINAL home

The official knockout pairing should be configured, not hard-coded. For example, a rule set can define A1 vs B2, C1 vs D2, and so on. The exact pairing order should be confirmed with the tournament rules before production.

10.4 Third-place match

Support a boolean or bracket-rule option for third-place matches. This keeps the engine compatible with either a final-only ending or a final-plus-third-place format.

11. API Design

11.1 Public API

GET /api/competitions/:competitionId
 GET /api/competitions/:competitionId/sports
 GET /api/competitions/:competitionId/sports/:sportId
 GET /api/competitions/:competitionId/sports/:sportId/groups
 GET /api/competitions/:competitionId/sports/:sportId/standings
 GET /api/competitions/:competitionId/sports/:sportId/matches
 GET /api/competitions/:competitionId/sports/:sportId/knockout
 GET /api/matches/:matchId
 GET /api/competition-teams/:teamId

11.2 Admin API

POST /api/admin/competitions
 PATCH /api/admin/competitions/:id
 POST /api/admin/competitions/:id/sports
 PATCH /api/admin/competition-sports/:id
 POST /api/admin/competition-sports/:id/teams
 POST /api/admin/competition-sports/:id/generate-groups
 PATCH /api/admin/competition-teams/:id/group
 POST /api/admin/competition-sports/:id/confirm-groups
 POST /api/admin/competition-sports/:id/generate-fixtures

```

PATCH /api/admin/matches/:id
POST /api/admin/matches/:id/start
POST /api/admin/matches/:id/result
POST /api/admin/matches/:id/overtime
POST /api/admin/matches/:id/super-over
POST /api/admin/matches/:id/penalties
POST /api/admin/matches/:id/postpone
POST /api/admin/competition-sports/:id/generate-knockout

```

11.3 API principles

- Validate all request bodies with Zod 4.
- Use consistent data/meta response envelopes.
- Return 409 for stale or conflicting competition mutations.
- Do not expose order IDs, payment data, user emails, payment codes, or audit metadata on public endpoints.
- Never trust client-supplied team relationships or winner IDs without server validation.

12. TypeScript / Backend Code Structure

```

src/competition/
  competition.routes.ts
  competition.service.ts
  competition-rules.ts
  competition-sport.service.ts
  team.service.ts
  group.service.ts
  group-draw.service.ts
  fixture.service.ts
  match.service.ts
  match-result.service.ts
  match-correction.service.ts
  standings.service.ts
  tiebreaker.service.ts
  qualification.service.ts
  knockout.service.ts
  bracket.service.ts
  competition.types.ts
  competition.schemas.ts

```

Keep rules functions pure where possible. Route handlers should orchestrate services, services should manage transactions and persistence, and rules should perform deterministic tournament calculations without HTTP dependencies.

13. Admin Dashboard

```

Dashboard
Orders
Payments
Teams
Events
-----

```

Championship
 Overview
 Futsal
 Cricksal
 Basketball

Settings

13.1 Championship overview

Sport	Companies	Groups	Group Fixtures	Status
Futsal	32	8	48	Setup / Live / Complete
Cricksal	20	4	40	Setup / Live / Complete
Basketball	16	4	24	Setup / Live / Complete

13.2 Competition health

Futsal
 Teams: 32/32
 Groups: 8/8
 Fixtures: 48
 Completed: 23
 Live: 2
 Postponed: 1
 Unresolved: 0

13.3 Live scoring screen

MATCH #24

ABC Bank [2]

VS

XYZ Ltd [1]

[Start] [+1] [-1] [Complete]

For basketball, expose +1/+2/+3 controls. For cricksal, expose the level of scorekeeping that RONB actually needs. Keep the primary scoring action one or two taps away.

14. Public Championship Website

Use the existing vanilla JavaScript ES-module approach. There is no need to introduce a new frontend framework for the first version.

/championship
 /championship/futsal
 /championship/cricksal
 /championship/basketball
 /championship/futsal/groups/a
 /championship/futsal/standings
 /championship/futsal/matches
 /championship/futsal/knockout

/championship/match/<id>
 /championship/company/<id>

14.1 Sport navigation

[Futsal] [Cricksal] [Basketball]

Overview | Groups | Standings | Fixtures | Results | Knockout

14.2 Match card

GROUP A · MATCH 12

ABC BANK 2 - 1 XYZ LTD

FULL TIME

14.3 Company page

ABC BANK
 [Company Logo]

Futsal
 Group A · Position 1 · 7 Points

Results
 ABC Bank 2-1 XYZ
 ABC Bank 1-1 DEF

If a company participates in multiple sports, its company page can expose Futsal, Cricksal, and Basketball entries while the competition entry remains sport-specific.

15. Live Updates, Scheduling, and Operations

Start with polling rather than introducing WebSockets or SSE. A 3-5 second refresh for live matches and a 10-20 second refresh for standings is sufficient for the first event and aligns with the existing simple frontend/backend architecture.

Store scheduled_at, venue, and court directly on matches for the first release. If multiple courts are heavily scheduled, introduce explicit competition locations/courts later and validate overlapping bookings.

15.1 Postponements

Never delete a postponed fixture. Keep the match identity and change the status to postponed, record the new schedule, and retain the original change in audit history.

15.2 Walkovers and forfeits

Plan for result_type values such as normal, walkover, forfeit, and abandoned. Whether those outcomes count as a particular score or carry special standings treatment should be a configured tournament rule, not a frontend assumption.

16. Security and Permissions

Action	Staff	Super Admin
View competition	Yes	Yes
Score live match	Yes	Yes
Postpone/cancel	Yes	Yes
Correct completed result	Limited	Yes
Create/configure competition	No	Yes
Generate groups/fixtures	No	Yes
Generate knockout	No	Yes
Reopen stage / destructive correction	No	Yes

Use the existing admin session and role system. High-impact changes such as completed-result corrections, stage reopening, or bracket-affecting edits should require super_admin plus a reason.

17. Migration Plan

```
011_competitions.sql
012_competition_sports.sql
013_competition_groups.sql
014_competition_teams.sql
015_matches.sql
016_match_periods.sql
017_competition_qualifiers.sql
018_match_result_versions.sql
019_competition_indexes.sql
020_competition_constraints.sql
```

Continue the existing migration ledger and advisory-lock approach. Keep each migration small enough to diagnose safely, but there is no technical requirement to split tables if the project prefers fewer files.

17.1 Existing events index cleanup

The current schema has a unique index on events(active). If the intended rule is "only one active event", that should be represented as a partial unique index on active=true, rather than uniqueness on both boolean values. This should be fixed in a migration before the championship becomes multi-event over time.

```
CREATE UNIQUE INDEX one_active_event
ON events(active)
WHERE active = true;
```

18. Testing Strategy

18.1 Unit tests

- 4-team group -> 6 fixtures.
- 5-team group -> 10 fixtures.
- Futsal win/draw/loss points -> 3/1/0.
- Cricksal win/loss points -> 2/0.
- Basketball win/loss points -> 2/0.
- Futsal 2-2 -> draw; Cricksal tie -> Super Over required; Basketball tie -> overtime required.
- Tie-break order, including three-way ties.

- Qualification counts and seed generation.
- Knockout bracket sizes and winner propagation.

18.2 Integration tests

- Group generation is idempotent.
- Fixture generation is idempotent and duplicate-safe.
- Result submission is transactional.
- A stale result submission returns 409 Conflict.
- Corrected results update standings without deleting history.
- Knockout winner propagation occurs after a completed source match.
- Concurrent match result updates resolve safely.

18.3 Playwright journeys

Admin:

login -> championship -> import companies -> groups -> fixtures -> start match -> result -> standings -> knockout -> final

Public:

championship -> sport -> group -> standings -> match -> result -> bracket

Test desktop and narrow mobile widths. Scores and standings are event-day content, so mobile usability is a primary requirement rather than a final polish item.

19. Full Tournament Simulation

Before production, create a disposable tournament with the exact target sizes: 32 futsal, 20 cricksal, and 16 basketball companies.

21. Generate groups for all three sports.
22. Generate all 112 group fixtures.
23. Simulate legal results for every fixture.
24. Verify standings and tie-breaks.
25. Generate the correct qualifiers.
26. Generate the knockout brackets.
27. Simulate every knockout round.
28. Verify final winners and champion status.
29. Repeat the simulation many times, including edge cases such as ties and corrections.

Primary QA goal

The system should be able to run an entire championship from confirmed entries to champion without a human manually editing standings or bracket winners.

20. Production Rollout

30. Back up the production database.
31. Run and verify migrations.
32. Create the championship in setup state.
33. Configure Futsal, Cricksal, and Basketball rules.

34. Import confirmed registrations.
35. Verify 32 / 20 / 16 company counts.
36. Generate and review groups.
37. Confirm groups.
38. Generate fixtures.
39. Verify 48 / 40 / 24 fixture counts.
40. Open the public championship pages.
41. Start event-day scoring and monitoring.

21. Event-Day Operating Workflow

Admin opens match

- > Start Match
- > update score
- > enter final score / resolution
- > Submit Result
- > backend validates + commits
- > standings / bracket update
- > public site reflects result

Keep the event-day screen focused. The person scoring a match should not need to understand the database or tournament configuration.

22. Recommended MVP Scope

- Competition creation and sport configuration.
- Confirmed-registration import into competition entries.
- Group generation, manual pre-fixture adjustments, and confirmation.
- Round-robin fixture generation.
- Match scheduling and live/completed states.
- Futsal draws; cricksal Super Over; basketball overtime.
- Automatic standings and configurable tie-breakers.
- Automatic qualification and knockout generation.
- Automatic bracket progression.
- Result corrections, audit logs, and concurrency protection.
- Admin scoring UI and public groups/standings/fixtures/results/knockout pages.
- Responsive mobile layout and Vitest + Playwright coverage.

23. V2 Candidates

- Company-wide historical statistics across events.
- Advanced player statistics.
- Push notifications and richer email automation.
- Full cricksal scorecards / NRR analytics if required.
- WebSockets or SSE for true push-based live updates.
- Advanced scheduling optimization.
- Automatic social media publishing and live commentary.

- AI-generated match analysis.

24. Final Implementation Checklist

- [] 01. Fix events active-index semantics.
- [] 02. Create competition migrations.
- [] 03. Create competition types and Zod schemas.
- [] 04. Create competition services and routes.
- [] 05. Import confirmed registrations as competition entries.
- [] 06. Implement reproducible group draw.
- [] 07. Implement manual group adjustment before fixture lock.
- [] 08. Implement round-robin fixture generation.
- [] 09. Implement match lifecycle.
- [] 10. Implement sport-specific result rules.
- [] 11. Implement standings and tie-break engine.
- [] 12. Implement qualification records.
- [] 13. Implement knockout bracket generation.
- [] 14. Implement automatic bracket progression.
- [] 15. Implement result correction history.
- [] 16. Integrate audit logs.
- [] 17. Add optimistic concurrency and row locking.
- [] 18. Build admin championship dashboard.
- [] 19. Build live scoring screen.
- [] 20. Build public championship pages.
- [] 21. Build mobile layouts.
- [] 22. Run unit/integration/concurrency tests.
- [] 23. Run full tournament simulation.
- [] 24. Run Playwright admin/public journeys.
- [] 25. Migrate production and verify counts before opening public competition.

25. Definition of Done

68 confirmed company entries
 -> 32 Futsal + 20 Cricksal + 16 Basketball
 -> 16 groups
 -> 112 group fixtures
 -> valid results
 -> final standings
 -> top 2 qualifiers from each group
 -> correct knockout brackets
 -> automatic progression
 -> sport-specific tie resolution
 -> finals
 -> champions

- No duplicate fixtures.
- No self matches.

- No invalid score states.
- No unresolved winners after a completed knockout match.
- No manually edited standings.
- No lost result history.
- No silent bracket corruption after corrections.
- Public scores and standings reflect authoritative backend data.

26. Open Rules to Confirm Before Coding the Final Bracket

The architecture above intentionally leaves several tournament-policy details configurable rather than inventing them: the exact top-two qualification confirmation, exact tie-break order for each sport, official knockout pairings, whether third-place matches exist, exact futsal knockout tie resolution procedure, exact cricksal standings statistic (such as NRR) and the handling of walkovers/forfeits/postponements.

Recommended next build step

Lock the tournament rule sheet first, then implement migrations and the pure rules engine before building the admin/public UI. This minimizes rewrites and makes the resulting system auditable.

Source basis: current RONB Events database schema supplied in this conversation. Key existing structures include events, sports, orders, order_items, team_players, admins, audit_logs, payment and session tables. Competition tables in this document are proposed additions; tournament rule details not yet finalized are explicitly marked as configurable or requiring confirmation.